

Wie die Daten gespeichert werden

Damit die Analyse in drei Jahren komplett anders aussehen darf, ohne dass ein einziger Messwert dabei verloren geht. Dieses Dokument beantwortet die Frage, welche Datenstruktur zur Lage passt — und begründet jede Entscheidung an einem Fall, der sonst schiefgeht.

Der eine Grundsatz, aus dem alles folgt

Gespeichert wird, was *beobachtet* wurde — nie, was daraus *gefolgert* wurde.

Jede Folgerung steckt voller Annahmen: eine Tara, eine Kaliber-Grenze, eine Reinigungsregel, ein Modell. Wer die Folgerung speichert, friert diese Annahmen mit ein — und wenn sich eine davon als falsch herausstellt, ist der Messwert verloren. Wer die Beobachtung speichert, kann sie in zehn Jahren mit einer völlig anderen Rechnung neu auswerten.

Das klingt selbstverständlich und ist es nicht. Es ist verführerisch, „34 kg Schimmel“ zu speichern, weil genau das gebraucht wird. Aber 34 kg ist bereits ein Ergebnis: Waage-Endstand minus Waage-Anfangsstand, minus Behälter-Tara, unter der Annahme, dass niemand zwischendurch geleert hat. Vier Annahmen in einer Zahl.

GEFOLGERT — VERLOREN

Der Palox als eine Zahl:

```
schimmel_messung kg = 34
```

Stellt sich später heraus, dass die Behälter-Tara 45 kg statt 40 kg ist, lässt sich diese Zeile nicht mehr korrigieren — der Rohwert existiert nicht mehr.

BEOBACHTET — REPARIERBAR

Der Palox als zwei Ablesungen:

```
palox_ablesung art = 'beginn', brutto = 112 art  
= 'ende', brutto = 146
```

Die 34 kg entstehen erst beim Auswerten. Ändert sich eine Annahme, ändert sich die Ableitung — die Beobachtung bleibt, wie sie war.

Der Prüfsatz für jede neue Spalte: *Könnte jemand diese Zeile in fünf Jahren mit einer völlig anderen Rechnung neu auswerten?* Lautet die Antwort nein, fehlt etwas.

Die drei Schichten

1 · Beobachtung

Was jemand gesehen, gewogen, gezählt oder geantwortet hat. Wird nie geändert, nie gelöscht.

unantastbar
wächst nur an

2 · Kontext

Taras, Kaliber-Grenzen, Sortierschemata, Reinigungsregeln — jeweils mit dem Zeitraum, in dem sie galten.

datiert
nie überschrieben

3 · Ableitung

Netto, Koeffizienten, Kurven, Kaskade, Ranking. Alles Ansichten und gespeicherte Auswertungen.

jederzeit wegwerfbar
aus 1 + 2 neu gebaut

Schicht 3 muss sich vollständig löschen und aus 1 und 2 neu erzeugen lassen. Genau das kann `setup.sql` heute schon — und diese Eigenschaft ist der eigentliche Schutz gegen eine Analyse, die sich ändert.

Wird die Analyse eines Tages ganz anders — ein anderes Verderbsmodell, eine andere Kaskade, eine ökonomische statt einer Massen-Rechnung —, dann fällt Schicht 3 weg und wird neu gebaut. Schicht 1 und 2 bleiben unberührt. Das ist der ganze Trick, und alles Weitere in diesem Dokument dient nur dazu, ihn wirklich zu halten.

1 · Kontext muss datiert sein — sonst ändert eine Korrektur die Vergangenheit

Das ist der unauffälligste und gefährlichste Fehler. Stammdaten werden bearbeitet, weil sie falsch waren — und damit ändert sich *rückwirkend* jede Auswertung, die je darauf beruhte, ohne dass es jemandem auffällt.

Ein Fall aus Deinen Antworten: Das **Sortierschema hängt am Käufer**, nicht an der Sorte. Coop will Kaliberbänder, Migros will Kisten ab acht Kilo, und beim nächsten Auftrag ist es wieder anders. Die Kaliber-Grenzen sind damit keine Eigenschaft der Sorte mehr. Werden sie trotzdem als ein Wert je Sorte gespeichert und beim nächsten Auftrag überschrieben, dann wird die Sortier-CSV vom Oktober plötzlich mit den Grenzen vom Januar klassiert — und der gemessene Ausschussanteil ändert sich, ohne dass ein einziger Kürbis anders gewogen wurde.

Die Regel: Jede Stammdaten-Zeile bekommt `gilt_ab`. Geändert wird nie eine Zeile, sondern es kommt eine neue dazu. Jede Beobachtung trägt ihren Zeitstempel und findet darüber den Kontext, der damals galt.

Das betrifft konkret: **Gebinde-Tara** (ändert jedes Netto rückwirkend), **Kaliber-Grenzen und Sortierschemata**, die **Palox-Tara** von 45 kg, und die **Reinigungsregeln** der CSV — die sind übrigens schon heute je Lauf gespeichert, und das war richtig.

2 · Antworten sind Messwerte

„War alles aus einer Charge?“ ist keine Bedienlogik, sondern eine **Aussage über die Verlässlichkeit einer anderen Messung** — und damit selbst ein Datum. Dasselbe gilt für „Sind die Ausschuss-Paletten leer?“, „War es wenigstens dieselbe Sorte?“, „Stand ein Sortierdatum auf den Kisten?“.

Diese Fragen werden sich ändern; es kommen neue dazu und alte fallen weg. Für jede eine Spalte anzulegen, hiesse mitten in der Saison die Datenbank umbauen. Deshalb bekommen sie eine eigene Tabelle mit Schlüssel und Wert:

```
auftrag_angabe(auftrag_id, schluessel, wert, ts, erfasser)
('eine_charge', false) · ('gleiche_sorte', true) · ('sortierdatum', '2026-10-14')
```

Bewusste Grenze: Diese Bauform ist nur für Antworten gedacht — für Dinge, über die nicht gerechnet und nicht verknüpft wird. Alles, was in einer Summe oder einer Verknüpfung landet, bleibt eine richtige Spalte in einer richtigen Tabelle. Ein Datenmodell, in dem *alles* Schlüssel und Wert ist, kann am Ende gar nichts mehr prüfen.

3 · Fremddaten kommen roh herein und werden nie überschrieben

Vier Quellen liefern von aussen: das **Erntejournal** (Wareneingang), die **Sortiermaschine** (CSV), künftig **Perigon** (Warenausgang) und vielleicht der **Klimacomputer**. Für alle gilt dasselbe Muster:

```
import_datei(id, quelle, datei_name, roh_ablage, pruefsumme,
             zeitraum_von, zeitraum_bis, eingelesen_ts, eingelesen_von)
```

Die Rohdatei wird unverändert abgelegt, die daraus gelesenen Zeilen zeigen auf ihre Herkunftsdatei. Drei Eigenschaften fallen dabei ab, und alle drei brauchst Du für den Perigon-Plan:

- **Zweimal einlesen schadet nicht.** Die Prüfsumme erkennt dieselbe Datei wieder. Wer unsicher ist, ob ein Export schon drin war, lädt ihn einfach nochmal.
- **Nachliefern und laufend einspeisen ist dasselbe.** Ob der ganze bisherige Saisonverlauf auf einmal kommt oder jede Woche ein Stück, ändert am Ablauf nichts.
- **Ein korrigierter Export ersetzt den alten, ohne ihn zu löschen.** Die neue Datei kommt dazu, die Zeilen der alten werden als überholt markiert. Was einmal drin war, bleibt nachvollziehbar — und wenn die Korrektur selbst falsch war, ist der Weg zurück offen.

Für Perigon heisst das konkret: *Schick mir die Vorlage, und ich baue den Import so, dass Du die Datei nur noch hochlädst.* Was die App aus den Spalten macht, ist dann Schicht 3 und jederzeit änderbar — die Datei selbst liegt unangetastet daneben.

4 · Jede Zeile weiss, woher sie kommt

In zwei Jahren wird jemand auf einen unplausiblen Wert stossen und fragen, wo der herkommt. Ohne Herkunft ist das eine Sackgasse.

Deshalb trägt jede Beobachtung: **wer** (Profil), **wann** (Serverzeit, nicht die Uhr des Handys), **auf welchem Weg** (App, Import oder direkt per SQL) und **mit welcher Programmfassung**. Das Letzte klingt übertrieben und ist es nicht: Wenn eine Maske drei Wochen lang einen Rechenfehler hatte, ist die Programmfassung die einzige Möglichkeit, hinterher genau die betroffenen Zeilen zu finden.

Was das für die Tabellen bedeutet

Konkret, aus Deinen Antworten abgeleitet.

TABELLE	WAS SICH ÄNDERT UND WARUM
palox_ablesung NEU	Ersetzt die gespeicherte Menge durch die einzelnen Ablesungen: Auftrag, Station, Zeitpunkt, Art (Beginn / Ende / zwischendurch), Bruttogewicht ab Waage . Die Menge entsteht beim Auswerten. Ein zwischenzeitliches Leeren ist damit eine dritte Ablesung, kein Sonderfall.
ausschuss_wiegung NEU	Statt einer geschätzten Kilozahl: Kategorie, Kistenzahl, Gebindeart, Bruttogewicht — dieselbe Bauform wie bei den Paletten. Das Netto ist Ableitung.
kaeuerer NEU	Coop, Migros und was sonst dazukommt. Die Arbeiter legen neue selbst an; das ist kein Stammdaten-Pflegefall, sondern Teil der Erfassung.
sortierschema + schema_kategorie NEU	Je (Sorte × Käufer) und mit <code>gilt_ab</code> . Entweder eine Mindestmenge je Kiste oder eine Liste von Kategorien mit Grenzen. Ein Auftrag hält fest, <i>welche</i> Fassung er verwendet hat — damit die Klassierung reproduzierbar bleibt, auch wenn beim nächsten Mal anders sortiert wird.
gebinde GEÄNDERT	Tara bekommt <code>gilt_ab</code> . Eine korrigierte Tara verändert sonst rückwirkend jedes Netto der Saison.
auftrag_angabe NEU	Die Antworten aus dem Abschluss-Ablauf, als Schlüssel und Wert.
import_datei NEU	Herkunft für alles von aussen: Journal, Sortiermaschine, Perigon, Klimacomputer.
klima_messung NEU, OPTIONAL	Zeitpunkt, Temperatur, Feuchte. Braucht heute niemand — aber wenn die Daten ohnehin da sind, kostet das Ablegen nichts, und später lässt sich damit eine temperaturabhängige Verdunstungsrate prüfen. Genau dafür ist Schicht 1 da: aufheben, was man noch nicht auswerten kann.
auftrag.art GEÄNDERT	Der neue Arbeitsgang <i>Fax</i> kommt dazu.
einstellung GEÄNDERT	Erfassungsbeginn, Palox-Tara (45 kg).

Was bewusst bleibt, wie es ist

WAS	WARUM ES SCHON RICHTIG IST
Die CSV wird roh gespeichert BLEIBT	Reinigung und Klassierung sind eine eigene Schicht darüber, jederzeit neu ausführbar. Das ist genau das Muster, das dieses Dokument für alles andere fordert — es war hier schon da.
Die Auswertung sind Ansichten BLEIBT	Kein Zwischenspeicher, den jemand von Hand pflegen müsste. Die gespeicherten Auswertungen sind nur Beschleunigung und werden auf Knopfdruck neu gerechnet.
Abbrechen statt löschen BLEIBT	Eine abgebrochene Arbeit verschwindet aus jeder Rechnung, aber nicht aus der Datenbank. Gelöscht ist ein Merkmal, keine Abwesenheit.
Die Chargennummer als Schlüssel BLEIBT	Eine undurchsichtige Nummer aus dem Journal, immun gegen Schreibweisen und Leerzeichen. Sie verbindet vier Systeme miteinander.
Leer ist nicht null BLEIBT	Eine fehlende Messung ist eine Lücke, keine Null. Wer beides verwechselt, rechnet mit erfundenen Nullen — und merkt es nie.

Die Ausstiegsklausel

Der härteste Test für „sauber gespeichert“ ist nicht, ob die eigene App damit umgehen kann, sondern ob **jemand anders** es kann — ohne diese App, ohne diese Datenbank, ohne mich.

Deshalb gehört ein vollständiger Export dazu: je Beobachtungstabelle eine Datei, dazu alle Rohdateien und eine Beschreibung, was jede Spalte bedeutet. Wer in fünf Jahren mit einem anderen Werkzeug neu rechnen will, braucht genau das und sonst nichts. Solange dieser Export existiert und vollständig ist, ist das Projekt nicht an seine eigene Software gefesselt.

DER SATZ, AN DEM SICH ALLES MESSEN LÄSST

Wenn morgen jemand käme und sagte: „*Euer Verderbsmodell ist Unsinn, ich rechne das ganz anders*“ — dann müsste er dafür **keine einzige neue Messung** brauchen. Er bekäme den Export, würde Schicht 3 weg und baute seine eigene. Alles, was auf dem Hof je beobachtet wurde, stünde ihm vollständig zur Verfügung.

Diesen Satz kann das Projekt heute schon fast halten. Was noch fehlt, steht in der Tabelle oben — und es ist überschaubar.

In welcher Reihenfolge

WAS	WARUM JETZT
1 Palox als Ablesungen, Ausschuss als Wägung, Antworten als Daten	Betrifft jede Erfassung ab sofort. Was hier heute falsch gespeichert wird, ist später nicht mehr zu retten.
2 Sortierschema je Käufer, mit Gültigkeit — samt der Maske, in der die Arbeiter es selbst eintragen	Ohne das werden Kaliber-Grenzen überschrieben und ändern rückwirkend die Vergangenheit.
3 Import für Fremddateien, Erfassungsbeginn	Sobald die Perigon-Vorlage da ist. Vorher lässt sich der Import nicht sinnvoll bauen.
4 Grundaussortierung vom Lagerverderb trennen; zu Kleine zum anderen Kanal	Ändert die Rechnung, nicht die Erfassung — geht also auch später, ohne dass Daten verloren gehen. Genau dafür ist die Schichtung da.
5 Export, Klimadaten	Wertvoll, aber nichts hängt daran fest.

Die Reihenfolge folgt einem einzigen Kriterium: **Was verliert Daten, wenn es fehlt?** Die Punkte 1 bis 3 tun das — jede Erfassung, die heute in der falschen Form abgelegt wird, ist verloren. Punkt 4 und 5 ändern nur, wie gerechnet wird, und das lässt sich jederzeit nachholen.